

Java Exception Handling

1. ClassNotFoundExceptionTest.java

`ClassNotFoundException` happens when Java tries to load a class by its name (as text) using `Class.forName()`, but no class with that name exists on the classpath. It's a **checked exception**, meaning the compiler forces you to handle it.

2. Step-wise Execution

1. Program starts, Scanner object is created to read input.
2. User is asked to enter a class name (e.g., "java.lang.String").
3. Program enters try block.
4. `Class.forName(name)` attempts to locate and load the class.
5. IF class exists -> "Class was found." is printed.
IF class missing -> `ClassNotFoundException` is thrown and caught,
"Class was not found." is printed.
6. finally block always runs -> "Class check completed."
7. Scanner is closed.

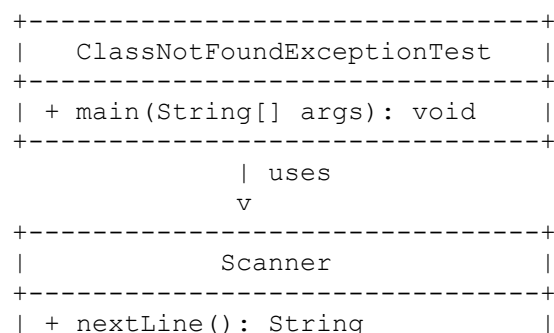
3. Algorithm

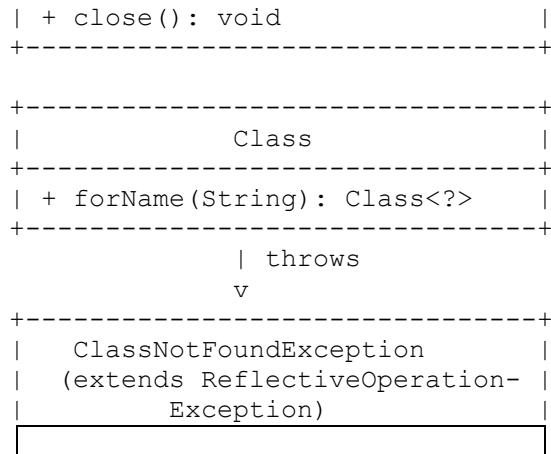
START

1. Create Scanner object
2. Prompt and read class name from user
3. TRY:
 - a. Call `Class.forName(name)`
 - b. IF successful, print "Class was found."
- CATCH `ClassNotFoundException`:
 - a. Print "Class was not found."
- FINALLY:
 - a. Print "Class check completed."
 - b. Close Scanner

STOP

4. UML Class Diagram





Code:

```
package testexception;

import java.util.Scanner;

public class ClassNotFoundExceptionTest {
    public static void main(String[] args) {
        //taking class name from user
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a class name: ");
        String name = sc.nextLine();

        try {
            //trying to find the class
            Class.forName(name);
            System.out.println("Class was found.");
        } catch (ClassNotFoundException exception) {
            //catch handles class not found
            System.out.println("Class was not found.");
        } finally {
            System.out.println("Class check completed.");
            sc.close();
        }
    }
}
```

```
}
```

Output:

```
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'testexception.ClassNotFoundExceptionTest'
Enter a class name: basics.Student
Class was found.
Class check completed.
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'testexception.ClassNotFoundExceptionTest'
Enter a class name: inheritance.file
Class was not found.
Class check completed.
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'testexception.ClassNotFoundExceptionTest'
Enter a class name: inheritance.Hybrid
Class was found.
Class check completed.
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'testexception.ClassNotFoundExceptionTest'
Enter a class name: hello
Class was not found.
Class check completed.
```

2. FileNotFoundExceptionTest.java

`FileNotFoundException` occurs when your program tries to open a file (to read from it) but the file does not exist at the given path, or it cannot be accessed. It is a **subclass of `IOException`**, so it must also be caught (or a parent `IOException` catch can handle it).

2. Step-wise Execution

1. Scanner reads a file name from the user.
2. Program enters try block.
3. `FileInputStream` tries to open the file.
4. IF file exists -> "File opened successfully." is printed, then file is closed.
IF file doesn't exist -> `FileNotFoundException` is thrown -> caught, "File was not found." is printed.
IF any other I/O error occurs -> caught by generic `IOException` block.
5. finally block runs -> "File check completed."
6. Scanner is closed.

3. Algorithm

START

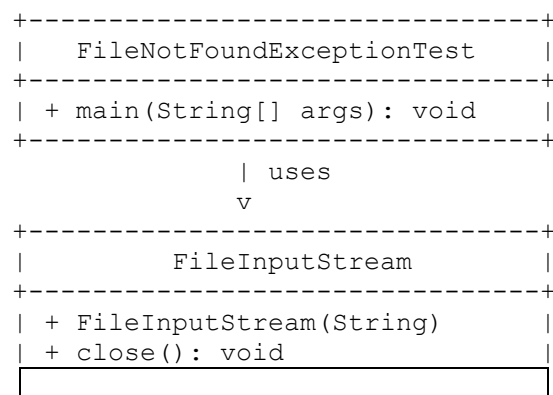
1. Create Scanner object
2. Prompt and read file name from user

```

3. TRY:
    a. Open FileInputStream with given file name
    b. Print "File opened successfully."
    c. Close the FileInputStream
CATCH FileNotFoundException:
    a. Print "File was not found."
CATCH IOException:
    a. Print "An I/O error occurred while accessing the file."
FINALLY:
    a. Print "File check completed."
    b. Close Scanner
STOP

```

4. UML Class Diagram



Code:

```

package testexception;

import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.IOException;
import java.util.Scanner;

public class FileNotFountExceptionTest {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a file name: ");
        String file = sc.nextLine();

        try {

```

```

        //trying to open file
        FileInputStream fileinput = new
FileInputStream(file);
        System.out.println("File opened successfully.");
        //every opened file need to close it after use
otherwise it will cause memory leak
        fileinput.close();
    } catch (FileNotFoundException exception) {
        //catch handles when file not found
        System.out.println("File was not found.");
    } catch (IOException exception) {
        //catch handles when any input or output error
occurs

        System.out.println("An I/O error occurred while
accessing the file.");
    } finally {
        System.out.println("File check completed.");
        sc.close();
    }
}
}
}

```

Output:

```

PS C:\Users\Admin\OneDrive\Documents\basicjava> c:: cd 'c:\Users\Admin\OneDrive\Documents\basicjava'; & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCo
deDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'testexception.FileNotFoundExceptionTest'
Enter a file name: c:\Users\Admin\OneDrive\Documents\basicjava\src\beyond\SamePackage.java
File opened successfully.
File check completed.
PS C:\Users\Admin\OneDrive\Documents\basicjava> c:: cd 'c:\Users\Admin\OneDrive\Documents\basicjava'; & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCo
deDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'testexception.FileNotFoundExceptionTest'
Enter a file name: c:\Users\Admin\OneDrive\Documents\basicjava\src\beyond\SamePackage.java
File opened successfully.
File check completed.
PS C:\Users\Admin\OneDrive\Documents\basicjava> c:: cd 'c:\Users\Admin\OneDrive\Documents\basicjava'; & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCo
deDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'testexception.FileNotFoundExceptionTest'
Enter a file name: c:\Users\Admin\OneDrive\Documents\basicjava\src\oops\LibraryManagement.java
File was not found.
File check completed.
PS C:\Users\Admin\OneDrive\Documents\basicjava> c:: cd 'c:\Users\Admin\OneDrive\Documents\basicjava'; & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCo
deDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'testexception.FileNotFoundExceptionTest'
Enter a file name: demo.java
File was not found.
File check completed.

```

IOException

^
|
FileNotFoundException

3. ArrayIndexOutOfBoundsExceptionTest.java

ArrayIndexOutOfBoundsException is thrown when you try to access an array element using an index that is negative or greater than/equal to the array's length. It is an **unchecked (runtime) exception**.

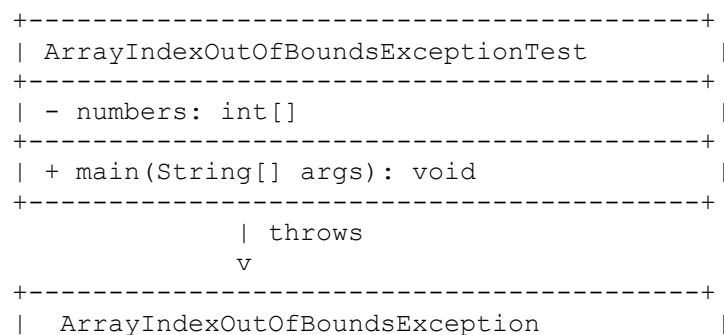
2. Step-wise Execution

1. An int array {10, 20, 30} is created (valid indices: 0,1,2).
2. User is asked to enter an index.
3. Program enters try block.
4. `sc.nextInt()` reads the index; `numbers[index]` accesses the array.
5. IF index is 0,1,or 2 -> value is printed.
 IF index is out of range -> ArrayIndexOutOfBoundsException is caught,
 "The array index is outside the valid range."
printed.
6. finally block runs -> "Array check completed."
7. Scanner is closed.

3. Algorithm

```
START
1. Declare int array numbers = {10, 20, 30}
2. Prompt and read index from user
3. TRY:
    a. Access numbers[index]
    b. Print the value
CATCH ArrayIndexOutOfBoundsException:
    a. Print "The array index is outside the valid range."
FINALLY:
    a. Print "Array check completed."
    b. Close Scanner
STOP
```

4. UML Class Diagram



```
(extends IndexOutOfBoundsException)
```

Code:

```
package testexception;

import java.util.Scanner;

public class ArrayIndexOutOfBoundsExceptionTest {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int[] numbers = {10, 20, 30};
        System.out.print("Enter an array index : ");

        try {
            //wrong index causes exception
            int index = sc.nextInt();
            System.out.println("Value: " + numbers[index]);
        } catch (ArrayIndexOutOfBoundsException exception) {
            //catch handles wrong index
            System.out.println("The array index is outside the
valid range.");
        } finally {
            //finally always runs
            System.out.println("Array check completed.");
            sc.close();
        }
    }
}
```

Output:

```

Array check completed.
PS C:\Program Files\Java\jdk-26.0.1\bin> java.exe --enable-preview -XX:+ShowCodeDetailsInExceptionMessages -cp C:\Users\Admin\OneDrive\Documents\basicjava\bin testexception.ArrayIndexOutOfBoundsExceptionTest
Enter an array index : 10
The array index is outside the valid range.
Array check completed.
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' --enable-preview -XX:+ShowCodeDetailsInExceptionMessages -cp 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' testexception.ArrayIndexOutOfBoundsExceptionTest
Enter an array index : 1
Value: 20
Array check completed.
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' --enable-preview -XX:+ShowCodeDetailsInExceptionMessages -cp 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' testexception.ArrayIndexOutOfBoundsExceptionTest
Enter an array index : 4
The array index is outside the valid range.
Array check completed.
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' --enable-preview -XX:+ShowCodeDetailsInExceptionMessages -cp 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' testexception.ArrayIndexOutOfBoundsExceptionTest
Enter an array index : 0
Value: 10
Array check completed.
PS C:\Users\Admin\OneDrive\Documents\basicjava>

```

4. IllegalArgumentExceptionTest.java

`IllegalArgumentException` is thrown manually (using `throw`) when a method receives an argument that is invalid or inappropriate for its logic — here, a negative age. It's an **unchecked exception** that signals bad input to a method.

2. Step-wise Execution

1. User enters age (integer).
2. Program enters try block and calls `setAge(age)`.
3. Inside `setAge()`:
 IF `age < 0` -> `throw new IllegalArgumentException("Age cannot be negative.")`
 ELSE -> `print "Age set to <age>"`
4. IF exception thrown -> caught in main, `exception.getMessage()` is printed.
5. finally block runs -> "Age check completed."
6. Scanner is closed.

3. Algorithm

START

1. Prompt and read age from user

2. TRY:

- a. Call `setAge(age)`

i. IF `age < 0`, `THROW IllegalArgumentException("Age cannot be negative.")`

- ii. ELSE `print "Age set to " + age`

CATCH `IllegalArgumentException`:

- a. `Print exception.getMessage()`

FINALLY:

- a. `Print "Age check completed."`
- b. `Close Scanner`

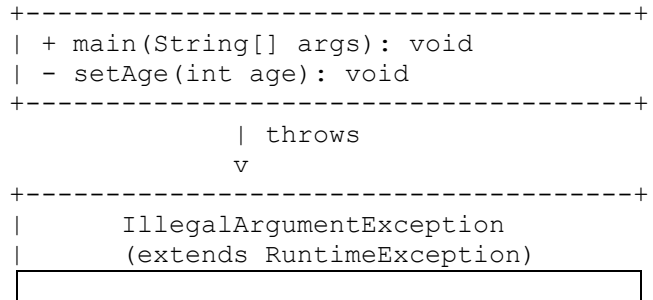
STOP

4. UML Class Diagram

```

+-----+
|      IllegalArgumentExceptionTest      |

```



Code:

```

package testexception;

import java.util.Scanner;

public class IllegalArgumentExceptionTest {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter your age: ");
        int age = sc.nextInt();

        try {
            setAge(age);
        } catch (IllegalArgumentException exception) {
            //catch handles wrong argument
            System.out.println(exception.getMessage());
        } finally{
            System.out.println("Age check completed.");
            sc.close();
        }
    }

    private static void setAge(int age) {
        if (age < 0) {
            //negative age is not allowed
            throw new IllegalArgumentException("Age cannot be
negative.");
        }
    }
}

```

```

    }
    System.out.println("Age set to " + age);
}
}

```

Output:

```

PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'testexception.IllegalArgumentExceptionTest'
Enter your age: 23
Age set to 23
Age check completed.
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'testexception.IllegalArgumentExceptionTest'
Enter your age: -81
Age cannot be negative.
Age check completed.
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'testexception.IllegalArgumentExceptionTest'
Enter your age: 102
Age set to 102
Age check completed.
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'testexception.IllegalArgumentExceptionTest'
Enter your age: #
Exception in thread "main" java.util.InputMismatchException
    at java.base/java.util.Scanner.throwFor(Scanner.java:977)
    at java.base/java.util.Scanner.next(Scanner.java:1632)
    at java.base/java.util.Scanner.nextInt(Scanner.java:2297)
    at java.base/java.util.Scanner.nextInt(Scanner.java:2251)
    at testexception.IllegalArgumentExceptionTest.main(IllegalArgumentExceptionTest.java:9)
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'testexception.IllegalArgumentExceptionTest'
Enter your age: 5
Age set to 5
Age check completed.

```

5. IllegalStateExceptionTest.java

`IllegalStateException` is thrown when a method is called at a time when the object/program is not in the right "state" to perform that action — here, when the country field is blank. It's an **unchecked exception**.

2. Step-wise Execution

1. User enters a country name (can be blank).
2. Static field 'country' stores the value.
3. Program enters try block and calls `showCountry()`.
4. Inside `showCountry()`:
 IF `country.isBlank()` -> throw new `IllegalStateException("No country was entered.")`
 ELSE -> print "You selected: <country>"
5. IF exception thrown -> caught in main, `exception.getMessage()` is printed.

6. finally block runs -> "Country check completed."
7. Scanner is closed.

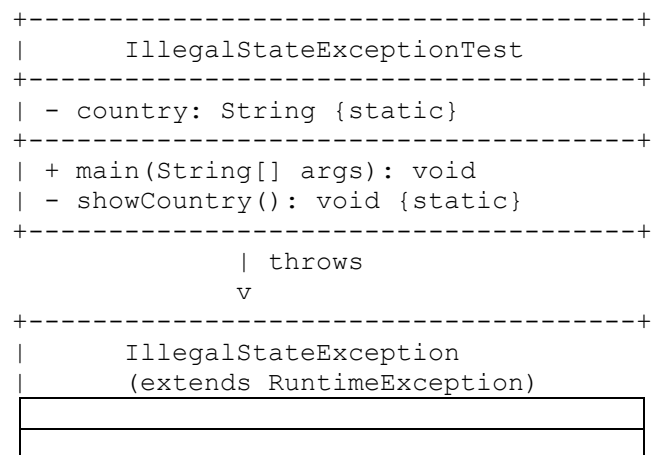
3. Algorithm

START

1. Prompt and read country from user, store in static field
2. TRY:
 - a. Call showCountry()
 - i. IF country is blank, THROW IllegalStateException("No country was entered.")
 - ii. ELSE print "You selected: " + country
- CATCH IllegalStateException:
 - a. Print exception.getMessage()
- FINALLY:
 - a. Print "Country check completed."
 - b. Close Scanner

STOP

4. UML Class Diagram



Code:

```

package testexception;

import java.util.Scanner;

public class IllegalStateExceptionTest {
    private static String country;

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
    }
  
```

```

        System.out.print("Enter a country: ");
        country = sc.nextLine();

        try {
            showCountry();
        } catch (IllegalStateException exception) {
            System.out.println(exception.getMessage());
        } finally {
            System.out.println("Country check completed.");
            sc.close();
        }
    }

    private static void showCountry() {
        if (country.isBlank()) {
            throw new IllegalStateException("No country was
entered.");
        }
        System.out.println("You selected: " + country);
    }
}

```

Output:

```

PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'testexception.IllegalStateExceptionTest'
Enter a country: INDIA
You selected: INDIA
Country check completed.
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'testexception.IllegalStateExceptionTest'
Enter a country: qwerty
You selected: qwerty
Country check completed.
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'testexception.IllegalStateExceptionTest'
Enter a country:
No country was entered.
Country check completed.
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'testexception.IllegalStateExceptionTest'
Enter a country: norway
You selected: norway
Country check completed.

```

6. InterruptedExceptionTest.java

`InterruptedException` is thrown when a thread that is sleeping, waiting, or paused (via `Thread.sleep()`) is interrupted by another thread while it is inactive. It's a **checked exception** common in multithreading.

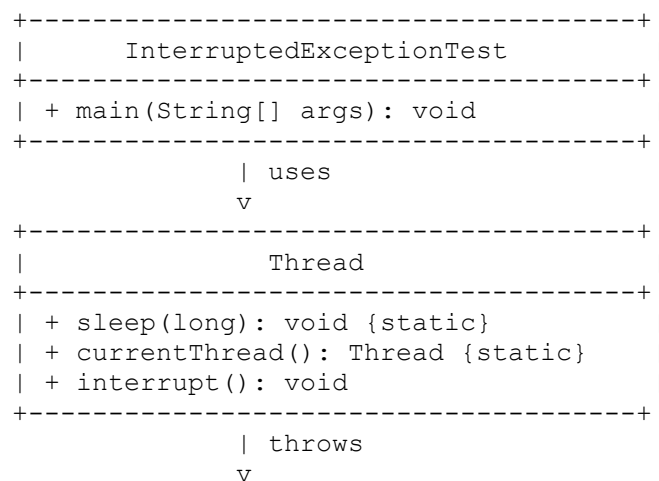
2. Step-wise Execution

1. User enters sleep time in milliseconds.
2. Program enters try block.
3. `Thread.sleep(time)` pauses the current thread.
4. IF sleep completes normally -> "Sleep completed." is printed.
IF thread is interrupted -> `InterruptedException` is caught,
`Thread.currentThread().interrupt()` re-flags interruption,
"The thread was interrupted." is printed.
5. finally block runs -> "Thread check completed."
6. Scanner is closed.

3. Algorithm

```
START
1. Prompt and read sleep time (ms) from user
2. TRY:
    a. Call Thread.sleep(time)
    b. Print "Sleep completed."
CATCH InterruptedException:
    a. Re-interrupt current thread
    b. Print "The thread was interrupted."
FINALLY:
    a. Print "Thread check completed."
    b. Close Scanner
STOP
```

4. UML Class Diagram



+-----+	
	InterruptedException
	(extends Exception)

Code:

```
package testexception;

import java.util.Scanner;

public class InterruptedExceptionTest {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter sleep time in milliseconds: ");
        int time = sc.nextInt();

        Thread mainThread = Thread.currentThread();

        Thread interrupter = new Thread(() -> {
            try {
                Thread.sleep(1000);
                mainThread.interrupt();
            } catch (InterruptedException exception) {
                Thread.currentThread().interrupt();
            }
        });

        interrupter.start();

        try {
            System.out.println("Main thread sleeping for " +
time + " milliseconds...");
            Thread.sleep(time);
            System.out.println("Sleep completed.");
        } catch (InterruptedException exception) {
            Thread.currentThread().interrupt();
        }
    }
}
```

```

        System.out.println("The thread was interrupted.");
    } finally {
        System.out.println("Thread check completed.");
        sc.close();
    }
}
}

```

Output:

```

PS C:\Users\Admin\OneDrive\Documents\basicjava> c:: cd 'c:\Users\Admin\OneDrive\Documents\basicjava' ; & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'c:\Users\Admin\OneDrive\Documents\basicjava\bin' 'testexception.InterruptedExceptionTest'
Enter sleep time in milliseconds: 45679
Main thread sleeping for 45679 milliseconds...
The thread was interrupted.
Thread check completed.
PS C:\Users\Admin\OneDrive\Documents\basicjava> c:: cd 'c:\Users\Admin\OneDrive\Documents\basicjava' ; & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'c:\Users\Admin\OneDrive\Documents\basicjava\bin' 'testexception.InterruptedExceptionTest'
Enter sleep time in milliseconds: 3321
Main thread sleeping for 3321 milliseconds...
The thread was interrupted.
Thread check completed.
PS C:\Users\Admin\OneDrive\Documents\basicjava> c:: cd 'c:\Users\Admin\OneDrive\Documents\basicjava' ; & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'c:\Users\Admin\OneDrive\Documents\basicjava\bin' 'testexception.InterruptedExceptionTest'
Enter sleep time in milliseconds: 230
Main thread sleeping for 230 milliseconds...
Sleep completed.
Thread check completed.
PS C:\Users\Admin\OneDrive\Documents\basicjava> c:: cd 'c:\Users\Admin\OneDrive\Documents\basicjava' ; & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'c:\Users\Admin\OneDrive\Documents\basicjava\bin' 'testexception.InterruptedExceptionTest'
Enter sleep time in milliseconds: 10000
Main thread sleeping for 10000 milliseconds...
The thread was interrupted.
Thread check completed.
PS C:\Users\Admin\OneDrive\Documents\basicjava> c:: cd 'c:\Users\Admin\OneDrive\Documents\basicjava' ; & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'c:\Users\Admin\OneDrive\Documents\basicjava\bin' 'testexception.InterruptedExceptionTest'
Enter sleep time in milliseconds: 1000
Main thread sleeping for 1000 milliseconds...
The thread was interrupted.
Thread check completed.

```

7. IOExceptionTest.java

`IOException` is a general **checked exception** representing a problem during input/output operations. In this program it is thrown manually when the user enters no data, simulating a "read failure."

2. Step-wise Execution

1. User enters some text data (can be empty).
2. Program enters try block and calls `readData(data)`.
3. Inside `readData()`:


```

      IF data.isEmpty() -> throw new IOException("No data was entered.")
      ELSE               -> print "Data: <data>"
      
```

4. IF exception thrown -> caught in main, message printed with prefix.
5. finally block runs -> "Input/output check completed."
6. Scanner is closed.

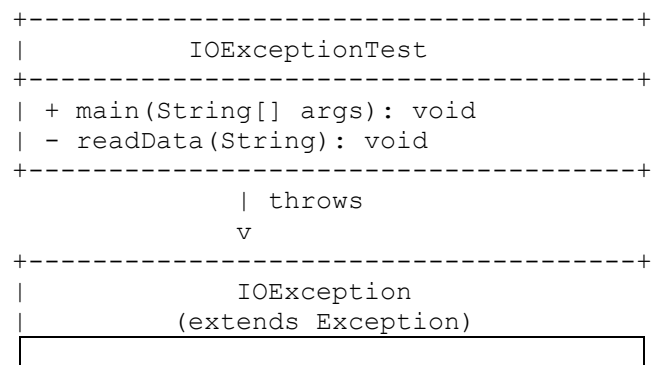
3. Algorithm

START

1. Prompt and read data string from user
2. TRY:
 - a. Call readData(data)
 - i. IF data is empty, THROW IOException("No data was entered.")
 - ii. ELSE print "Data: " + data
- CATCH IOException:
 - a. Print "Input/output error: " + exception.getMessage()
- FINALLY:
 - a. Print "Input/output check completed."
 - b. Close Scanner

STOP

4. UML Class Diagram



Code:

```
package testexception;
import java.io.IOException;
import java.util.Scanner;

public class IOExceptionTest {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter data to read: ");
        String data = sc.nextLine();
    }
}
```

```
        try {
            readData(data);
        } catch (IOException exception) {
            //catch handles input output problem
            System.out.println("Input/output error: " +
exception.getMessage());
        } finally {
            System.out.println("Input/output check
completed.");
            sc.close();
        }
    }

    private static void readData(String data) throws
IOException {
        if (data.isEmpty()) {
            //empty data causes input output exception
            throw new IOException("No data was entered.");
        }
        System.out.println("Data: " + data);
    }
}
```

Output:

```

PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' --enable-preview -XX:+ShowCodeDetailsInExceptionMessages -cp 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'testexception.IOExceptionTest'
Enter data to read:
Input/output error: No data was entered.
Input/output check completed.
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' --enable-preview -XX:+ShowCodeDetailsInExceptionMessages -cp 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'testexception.IOExceptionTest'
Enter data to read: 45463
Data: 45463
Input/output check completed.
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' --enable-preview -XX:+ShowCodeDetailsInExceptionMessages -cp 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'testexception.IOExceptionTest'
Enter data to read: all is well
Data: all is well
Input/output check completed.
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' --enable-preview -XX:+ShowCodeDetailsInExceptionMessages -cp 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'testexception.IOExceptionTest'
Enter data to read: $%#@
Data: $%#@
Input/output check completed.
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' --enable-preview -XX:+ShowCodeDetailsInExceptionMessages -cp 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'testexception.IOExceptionTest'
Enter data to read:
Input/output error: No data was entered.
Input/output check completed.
PS C:\Users\Admin\OneDrive\Documents\basicjava>

```

8. NullPointerExceptionTest.java

`NullPointerException` occurs when your code tries to use an object reference that is `null` (points to nothing) — such as calling a method on it. It's one of the most common **unchecked exceptions** in Java.

2. Step-wise Execution

1. User enters a name, or leaves it empty.
2. IF the entered text is empty -> name is manually set to `null`.
3. Program enters try block.
4. `name.length()` is called.
5. IF name is not `null` -> length is printed.
IF name is `null` -> `NullPointerException` is caught,
"Name cannot be null." is printed.
6. finally block runs -> "Name check completed."
7. Scanner is closed.

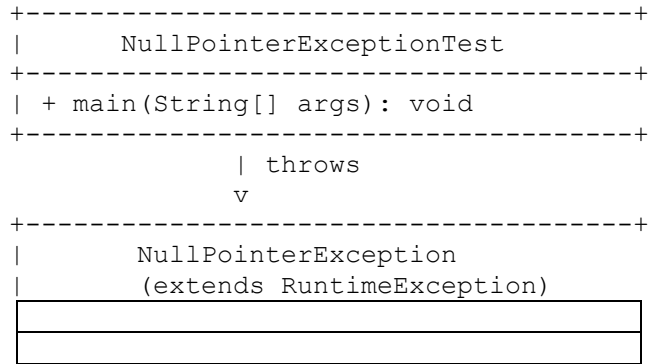
3. Algorithm

START

1. Prompt and read name from user
2. IF name is empty string, SET name = `null`
3. TRY:
 - a. Call `name.length()`
 - b. Print "Name length: " + result
- CATCH `NullPointerException`:
 - a. Print "Name cannot be null."
- FINALLY:
 - a. Print "Name check completed."
 - b. Close Scanner

STOP

4. UML Class Diagram



Code:

```

package testexception;

import java.util.Scanner;

public class NullPointerExceptionTest {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter your name or leave it empty:
");
        String name = sc.nextLine();

        //empty name is changed to null
        if (name.isEmpty()) {
            name = null;
        }

        try {
            //exception occurs when name is null
            System.out.println("Name length: " +
name.length());
        } catch (NullPointerException exception) {
            //catch handles the exception
            System.out.println("Name cannot be null.");
        } finally{
            System.out.println("Name check completed.");
        }
    }
}

```

```

        sc.close();
    }
}
}
}

```

Output:

```

PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'testexception.NullPointerExceptionTest'
Enter your name or leave it empty: susmitha
Name length: 8
Name check completed.
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'testexception.NullPointerExceptionTest'
Enter your name or leave it empty:
Name cannot be null.
Name check completed.
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'testexception.NullPointerExceptionTest'
Enter your name or leave it empty: null
Name length: 4
Name check completed.
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'testexception.NullPointerExceptionTest'
Enter your name or leave it empty: Nitrogen
Name length: 8
Name check completed.

```

9. NumberFormatExceptionTest.java

NumberFormatException is thrown when you try to convert a String into a number (using Integer.parseInt(), for example) but the String's content is not actually a valid number format. It's an **unchecked exception** and a subclass of IllegalArgumentException.

2. Step-wise Execution

1. User enters some text.
2. Program enters try block.
3. Integer.parseInt(text) attempts to convert text to an int.
4. IF text is a valid number -> number is printed.
IF text is not a valid number -> NumberFormatException is caught,
"The text is not a valid integer."
printed.
5. finally block runs -> "Number check completed."
6. Scanner is closed.

3. Algorithm

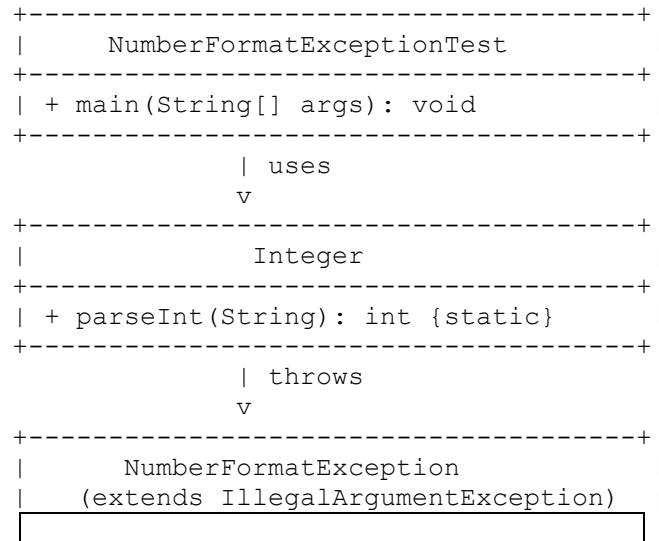
- START
1. Prompt and read text from user
 2. TRY:

```

    a. Call Integer.parseInt(text)
    b. Print "Number: " + result
CATCH NumberFormatException:
    a. Print "The text is not a valid integer."
FINALLY:
    a. Print "Number check completed."
    b. Close Scanner
STOP

```

4. UML Class Diagram



Code:

```

package testexception;

import java.util.Scanner;

public class NumberFormatExceptionTest {
    public static void main(String[] args) {
        //taking number as text
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a whole number: ");
        String text = sc.nextLine();

        try {
            //converting text into number

```

```

        int n = Integer.parseInt(text);
        System.out.println("Number: " + n);
    } catch (NumberFormatException exception) {
        //catch handles wrong number format
        System.out.println("The text is not a valid
integer.");
    } finally {
        System.out.println("Number check completed.");
        sc.close();
    }
}
}

```

Output:

```

PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'testexception.NumberFormatExceptionTest'
Enter a whole number: 32452
Number: 32452
Number check completed.
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'testexception.NumberFormatExceptionTest'
Enter a whole number: -35568
Number: -35568
Number check completed.
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'testexception.NumberFormatExceptionTest'
Enter a whole number: @ma
The text is not a valid integer.
Number check completed.
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'testexception.NumberFormatExceptionTest'
Enter a whole number: qwe
The text is not a valid integer.
Number check completed.
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'testexception.NumberFormatExceptionTest'
Enter a whole number: 12.5
The text is not a valid integer.
Number check completed.

```

10. SQLExceptionTest.java

`SQLException` is a **checked exception** that represents an error while interacting with a database (e.g., a bad query, connection failure, or invalid data). Here it is simulated manually when the query text is empty.

2. Step-wise Execution

1. User enters a query name (can be empty).

2. Program enters try block and calls executeQuery(query).
3. Inside executeQuery():
 - IF query.isEmpty() -> throw new SQLException("The query name is empty.")
 - ELSE -> print "Query received: <query>"
4. IF exception thrown -> caught in main, message printed with prefix.
5. finally block runs -> "Database check completed."
6. Scanner is closed.

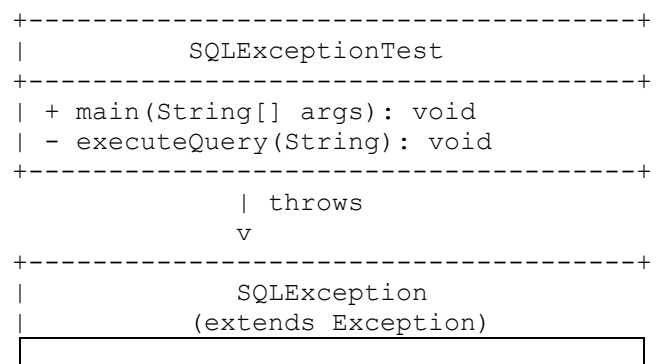
3. Algorithm

START

1. Prompt and read query string from user
2. TRY:
 - a. Call executeQuery(query)
 - i. IF query is empty, THROW SQLException("The query name is empty.")
 - ii. ELSE print "Query received: " + query
- CATCH SQLException:
 - a. Print "Database error: " + exception.getMessage()
- FINALLY:
 - a. Print "Database check completed."
 - b. Close Scanner

STOP

4. UML Class Diagram



Code:

```

package testexception;

import java.sql.SQLException;
import java.util.Scanner;

public class SQLExceptionTest {
    public static void main(String[] args) {
  
```

```

        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a query name: ");
        String query = sc.nextLine();

        try {
            executeQuery(query);
        } catch (SQLException exception) {
            //catch handles database problem
            System.out.println("Database error: " +
exception.getMessage());
        } finally {
            System.out.println("Database check completed.");
            sc.close();
        }
    }

    private static void executeQuery(String query) throws
SQLException {
        if (query.isEmpty()) {
            //empty query causes sql exception
            throw new SQLException("The query name is empty.");
        }
        System.out.println("Query received: " + query);
    }
}

```

Output:

```

PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'testexception.SQLExceptionTest'
Enter a query name: 1234
Query received: 1234
Database check completed.
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'testexception.SQLExceptionTest'
Enter a query name: wewtyt
Query received: wewtyt
Database check completed.
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'testexception.SQLExceptionTest'
Enter a query name:
Database error: The query name is empty.
Database check completed.
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'testexception.SQLExceptionTest'
Enter a query name: 12222223.4566
Query received: 12222223.4566
Database check completed.
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'testexception.SQLExceptionTest'
Enter a query name: @%$
Query received: @%$
Database check completed.
PS C:\Users\Admin\OneDrive\Documents\basicjava> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\basicjava\bin' 'testexception.SQLExceptionTest'
Enter a query name:
Database error: The query name is empty.
Database check completed.

```

Quick Reference Table

Program	Exception Class	Checked / Unchecked	Parent Class
ClassNotFoundExceptionTest	ClassNotFoundException	Checked	ReflectiveOperationException
FileNotFoundExceptionTest	FileNotFoundException	Checked	IOException
ArrayIndexOutOfBoundsExceptionTest	ArrayIndexOutOfBoundsException	Unchecked	IndexOutOfBoundsException
IllegalArgumentOutOfRangeExceptionTest	IllegalArgumentOutOfRangeException	Unchecked	RuntimeException
IllegalStateExceptionTest	IllegalStateException	Unchecked	RuntimeException
InterruptedExceptionTest	InterruptedException	Checked	Exception
IOExceptionTest	IOException	Checked	Exception
NullPointerExceptionTest	NullPointerException	Unchecked	RuntimeException
NumberFormatExceptionTest	NumberFormatException	Unchecked	IllegalArgumentException
SQLExceptionTest	SQLException	Checked	Exception